

CSA5112 - Cryptography and Network Security

13. Diffie–Hellman Key Exchange

Aim: To implement the Diffie–Hellman Key Exchange algorithm for securely generating a shared secret key between two users over an insecure communication channel.

Algorithm:

1. Start.
2. Choose a prime number (**P**) and a primitive root (**G**).
3. Read the private keys of User A and User B.
4. Compute the public keys:
 - $A = G^a \text{ mod } P$
 - $B = G^b \text{ mod } P$
5. Exchange the public keys.
6. Compute the shared secret key:
 - User A: $K = B^a \text{ mod } P$
 - User B: $K = A^b \text{ mod } P$
7. Verify that both users obtain the same shared secret key.
8. Display the shared secret key.
9. Stop.

Code:

```
# Diffie-Hellman Key Exchange

# Publicly known values

P = int(input("Enter Prime Number (P): "))

G = int(input("Enter Primitive Root (G): "))

# Private keys

a = int(input("Enter Private Key of User A: "))
```

```
b = int(input("Enter Private Key of User B: "))

# Public keys

A = pow(G, a, P)

B = pow(G, b, P)

print("\nPublic Key of User A:", A)

print("Public Key of User B:", B)

# Shared Secret Keys

keyA = pow(B, a, P)

keyB = pow(A, b, P)

print("\nSecret Key Computed by User A:", keyA)

print("Secret Key Computed by User B:", keyB)

if keyA == keyB:

    print("\nShared Secret Key:", keyA)

else:

    print("\nKey Exchange Failed")
```

Sample Input:

```
Enter Prime Number (P): 23
Enter Primitive Root (G): 5
Enter Private Key of User A: 6
Enter Private Key of User B: 15
```

Sample Output:

```
Public Key of User A: 8
Public Key of User B: 19

Secret Key Computed by User A: 2
Secret Key Computed by User B: 2

Shared Secret Key: 2
```

Result:

The Diffie–Hellman Key Exchange algorithm was successfully implemented. Both users generated the same shared secret key without transmitting the private keys.